



COLLEGE OF ENGINEERING, CONSTRUCTION, AND LIVING SCIENCES

BACHELOR OF INFORMATION TECHNOLOGY (BIT)

ASSIGNMENT-1, SEMESTER 2, 2026

(SCRIPTING) AUTOMATION OF USER, PROCESS, AND PERMISSION MANAGEMENT IN A
COMPUTING SYSTEM

ID616001 OPERATING SYSTEMS CONCEPTS

LEVEL 6, CREDITS 15

VERSION 1.1

General Instructions

This assignment contributes 15% to your total course grade.

- This is an individual assessment
- All submissions must be your own original work. If you reference any external resources (online or offline), you must provide appropriate citations and credit.
- You may use online tools or websites (e.g., Google) to assist your work; however, all sources used must be properly acknowledged.
- **Collaboration is strictly prohibited.** All tasks must be completed independently.
- **Assignment Due Date:** 11th, September 2026, 11:59PM

Learning Outcomes

At the successful completion of this course, students will be able to:

1. Perform automated user and permissions management in a computing system.
2. Analyse and manage process, memory, storage, and network management modules in operating systems.

Instructions

In this assignment you will write a collection of Bash scripts to help automate general administrative tasks on an Ubuntu Linux server. The assignment comprises the following components:

- **Task 1:** Script to automate user creation and configuration of the user environment (30%).
- **Task 2:** Script to backup directories and upload the backup to a remote server (20%).
- **Documentation, Script Quality & Git:** Error handling, documentation, commenting, code modularity and git usage (20%).
- **Code defense - 30%**

1 Task 1 – Creating a User Environment (30%)

Write a script to automate the process of user creation and configuration of user environment. The script will ingest a file containing user-related information and create users accordingly (based on the information in the provided file). It should also perform configuration tasks such as creating secondary groups, shared folders, and setting permissions.

Core Functionality

The core functionality required for your Bash script is outlined below:

- The script must accept a CSV file (local or remote) containing user details needed to perform the following tasks:
 - Create each specified user while implementing best practices;
 - Set a default password for the user based on the user's birth date;
 - Create the required secondary groups (if specified and non-existent);
 - Assignment to the correct secondary groups (if specified);
 - Create the required shared folder (if specified and non-existent);
 - Create the required secondary group for the shared folder (if specified and non-existent);
 - Assignment of the user to a group to access the shared folder (if specified);
 - Create a symbolic link in each user's home directory to their shared folder.
 - Create a custom alias for sudo users to display the contents of their home directory
 - Enforce a password change at first login

1.1 Script Input File

As stated, your script must accept a CSV file containing user data, either via:

1. **Command-Line Arguments:** Must be able to handle two different command line arguments, either:
 - (a) A URI to a web-based CSV resource, e.g. <https://osc.op-bit.nz/share/users.csv>, **or**
 - (b) A local file system location e.g. [/home/<your-username>/files/users.csv](#)
2. **Interactive Prompt (if no arguments are provided):** Must be able to handle two different user inputs when no command-line argument has been supplied either:
 - (a) A URI to a web-based CSV resource, **or**
 - (b) A local file path.

File Validation:

- For local files, check that the file exists and is a valid, parseable CSV.
- For remote files: If the file is provided by via a URI , the script should download the file (to the same directory as the script), check file existence, that it is parse-able and then proceed with processing.

Once the input file is ingested, the script should parse the file and use it to create users and configure the environment on the local system. One sample file has been provided as an example of the structure of the input file format. The example files are available from:

<https://osc.op-bit.nz/share/users.csv>,

The contents of the example file are listed below:

```
e-mail,birth date,groups,sharedFolder
edsgen.dijkstra@tue.nl,1930/05/11,sudo,staff,/staffData
john.mccarthy@caltech.edu,1927/09/04,sudo,visitor,/visitorData
andrew.tanenbaum@vua.nl,1944/03/16,staff,/staffData
alan.turing@cam.ac.uk,1912/06/23,visitor,/visitorData
linus.torvalds@linux.org,1969/12/28,sudo,
bjarne.stroustrup@tamu.edu,1950/12/30,,/visitorData
ken.thompson@google.com,1973/02/04,sudo,visitor,
james.gosling@sun.com,1955/05/19,staff,/staffData
tim.berners-lee@mit.edu,1955/06/08,sudo,visitor,/visitorData
```

Carefully check the format of the provided file and use it as an example of what parameters your script must be able to handle. Each row of the sample file represents an individual user. The sample file contains a header that documents each column in the file. The available columns in the file are:

- The email address of the user
- The birth date of the user (in the format YYYY/MM/DD, for example, 1991/11/17)
- The secondary groups the user should be added to
- A shared folder that the user requires full access to (full rwx permissions)

1.2 User Environment Requirements

Username: Generate the username from the provided e-mail address. Use the following convention: the first letter of the surname (lowercase), followed by the full first name with its first letter in uppercase. Example: `linus.torvalds@linux.org` → `tLinus`.

If the generated username already exists on the system, skip creating that user and display a console notification.

Password: Generate the default password from the user's birth date in the format YYYYMM (year followed by month). Example: Birth date 1969/12/28 → password 196912.

Use the `date` command to extract date components, and configure the account so that the user must change their password upon first login (see `chage` command).

Shared Folder: Create the shared folder if it does not already exist, and set appropriate permissions. Treat shared folder paths as absolute paths.

To maintain permissions for all existing users with access (and not just the most recently added user), use additional groups where appropriate. The owner of the shared folder should be either `root` or the user running the script. Users outside the intended group should have no access.

Shared Folder Link: For each user with access to a shared directory, create a symbolic link named `shared` in their home folder pointing to the shared directory.

Alias: For each user with `sudo` access, create an alias named `myls` that lists all files in their home directory (including hidden files) with permissions shown. Store this alias in the correct file (e.g., `.bash_alias`) so that it is loaded automatically on every login.

Best Practice: Follow best practices for creating and configuring users, groups, shared folders, links, and aliases. For example:

- Use consistent home directory naming conventions and default shell settings for new users.
- Store aliases in `.bash_alias` to ensure they persist across sessions.

1.3 Script Output and Logging

Script Output: Your script should provide console output (stdout) for major events. This is to provide the user with information about what actions the script is performing. The following list provides a summary of the required output:

- Print information about the input file
- Print a dialog about the number of users to be added
- For each user, print a summary of the created user environment including (but not limited to): username, home directory, shared folder, shared folder link, alias
- Print information when a major error is encountered, including (but not limited to): user creation failed, shared folder creation failed, group creation failed, alias creation failed, link creation failed

Before the script starts creating users and setting up the user environment, it should present the user a dialogue that informs about the number of to-be-added users and asks for confirmation before proceeding. Try not to use too much console-based output, if it is too cluttered the user will not be able to easily track what is happening. Also, formatting console-based output using line breaks or indentation can greatly increase readability.

Logging Output: Your script should also provide functionality to output additional detailed information to a log file. This log file should be much more detailed than the console output, and should keep track of the success/failure of user creation and success/failure of user environment configuration. This is specified so that the user can review the log file to determine where anything went wrong when running the script. This log file should include date information that indicates when the script was run. Make sure you also choose a sensible name for the log file.

1.4 Script Error Checking

Your script should check for all possible errors and deal with them by presenting the user with information and also logging it. Carefully think about things that can possibly go wrong or cause problems during script execution. Acknowledge possible problems in the script, even if you do not address them – that way your tutor knows you have identified the error but have not fixed it.

1.5 Code Structure and Best Practice

Code Structure: Structure your code by moving repetitive code into functions. When implementing functions, use local variables where possible to avoid eventual side effects of global variable usage.

Code Comments: Provide sensible documentation through comments. Sensible implies that you don't necessarily comment each line of code but comment critical or complex operations that might be difficult for another user to understand quickly when reviewing your code. This way the reader should get quick insight into what happens where in your code.

1.6 Hints

The following hints may provide useful during development of your script:

- Start with a basic script that reads the input URI and local file
- Start by using print statements instead of actual commands
- Create a dummy user input file that only contains one user for testing
- Test added code thoroughly before adding any additional functionality
- Create a clean-up script that removes users, groups and shared folders so that you can quickly reset your system back to default to ease the testing process
- If you don't manage to get all parts of the code working, annotate sections that are missing code or functionality and describe what should have happened there
- Continually save code changes using git (discussed later)

2 Task 2 – Backup Script (20%)

For Task 2, you will write a script that compresses a given folder and uploads it to a remote location — essentially a backup script. The script should be able to back up the contents of any directory in the local file system.

2.1 Core Functionality

Your backup script must:

- Accept a directory path as a command-line argument. If no argument is provided, prompt the user for the path.
- Create a gzip-compressed tarball (`.tar.gz`) of the entire directory. The archive should be named after the `basename` of the directory (e.g., `/home/user/docs` → `docs.tar.gz`).
- Prompt the user for:
 - Remote server IP address or hostname
 - Port number (if not default SSH 22)
 - Target directory on the remote server
- Upload the compressed tarball to the remote host using `scp` over SSH with the details provided by the user or `rsync`.
- Validate that the local directory exists before creating the tarball, and verify that the upload was successful.
- Name your backup files with a timestamp using the format: `backup-YYYY-MM-DD.tar.gz`.
- All submissions must be uploaded to your dedicated folder on the server:
`/var/assessments/<your-username>/`
- After uploading, verify your submission:

```
ssh <username>@<server-ip>
ls /var/assessments/<username>/
```
- Ensure the filename includes the correct timestamp. Incorrect or missing files may not be marked.

2.2 Error Checking

Your script should handle possible errors, including:

- Local directory does not exist
- Network issues or unreachable remote server
- Incorrect authentication credentials
- Invalid remote directory

For any error, display an informative message and safely terminate execution.

2.3 Hints

- Test your script on the provided VM before submitting.
- Practice creating gzip-compressed tarball archives from the command line.
- Test `scp` or `rsync` manually before automating.
- Verify the contents of the tarball to ensure all files are included.

3 Documentation and Git (20%)

This assignment includes a collection of formal requirements, including project structure, documentation, and the use of git for source code revision control and assignment submission.

3.1 Source Code Revision Control and Git Usage

You are required to use `git` for source code revision control and assignment submission. You may use any git platform of your choice (e.g., GitHub, GitLab, Bitbucket), but the repository must remain private and accessible only to yourself and your lecturer (Github account: OlayinkaOP).

Please provide a link to your git repository as soon as you start submitting code. Follow git best practices, including:

- Commit your code at logical and regular intervals.
 - Use descriptive commit messages that clearly summarize the changes made.
-

3.2 Project Structure

Your git repository and project source code should follow a clear and uniform structure. The repository should include:

```
README.md
BSA_Self_Assessment.txt
task1/
    (files related to Task 1)
task2/
    (files related to Task 2)
```

3.3 Project Documentation

Include documentation to accompany your assignment. Specifically, create a brief README file with the following requirements:

- Written in Markdown syntax.
- Named `README.md`.
- Located in the root directory of your git repository.

Your `README.md` file must contain:

- **Author Details:**
 - Your full name.
 - Your student code (login name).
 - Timestamp of the last date when changes were made.
- **Project Details for Each Script:**
 - Summary of the purpose of the script.
 - Pre-requisites for running the script.
 - Instructions on how to run the script, including example commands.

When writing your README, consider that the reader may have limited knowledge of Linux. Also, think ahead, imagine yourself six months from now revisiting these scripts. Your documentation should clearly explain the script's purpose, usage, and requirements.

Note: Other formal aspects, such as error handling, code commenting, and code modularity, will be assessed when reviewing your scripts for Task 1 and Task 2.

3.4 Assignment Submission

Before submission, thoroughly test all scripts to ensure proper functionality and error handling.

Submit your assignment via Microsoft Teams assessment submission channel as a tar file and embed the link to your Git repo. Ensure that your git repository is up-to-date and ready by the submission deadline. Your submission must include:

- A comprehensive report containing documentation and necessary screenshots for every successful execution of all required components. Include screenshots of `/etc/passwd` and `/etc/group`.
- The execution output of the `scp` command for Task 2.
- This document.
- The completed self-assessment worksheet (`A1_Self_Assessment.docx`) placed in the root directory of the submitted archive.

You may also include comments in the self-assessment worksheet for the marker.

4 Code Defense - 30%

To successfully pass the code defense, you must demonstrate both technical understanding and the ability to clearly communicate your solutions. The defense assesses not only whether the code runs correctly but also whether the you can justify your design choices, explain functionality, and reflect on potential issues.

Requirements to Pass Code Defense

Requirement	Description
Clarity	Clearly and confidently explain the logic and structure of your code.
Accuracy	Demonstrate correct use of Linux commands and scripting logic.
Error Awareness	Identify where failures may occur and how your script addresses them.
Logging	Justify the purpose and format of your audit logging.
Security Considerations	Explain permission settings such as <code>chmod</code> and <code>chown</code> .
Script Output	Accurately interpret output files and command-line results.
Reflection Knowledge	Be able to discuss your solutions clearly in your own words.